振動触覚提示チュートリアル

Python で振動波形をデザインし、振動子から提示するまでの 4 ステップ

作成日: 2026-08-23 ／ 対象: 研究室の新規メンバー・実験参加者

このチュートリアルでは、Python　で振動波形を作成し、オーディオ出力とアンプを介して振動子（振動触覚アクチュエータ）を駆動する方法を学びます。振動子はスピーカーと同じ原理で動くため、「音を鳴らすプログラム」がそのまま「振動を提示するプログラム」になります。以下の順に進めてください。

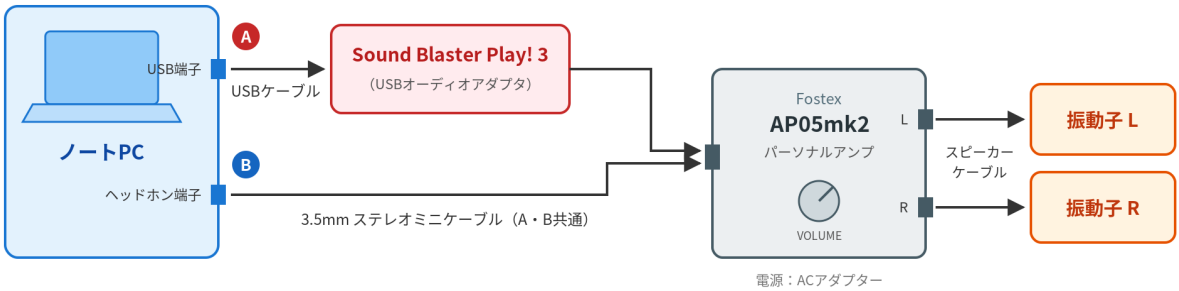
Step 1	`step1.py`	正弦波をデザインして 1 チャンネルで再生する
Step 2	`step2.py`	スレッドを使い、ステレオの L / R を同時に再生する
Step 2-2	`step2_2.py`	2 台のオーディオデバイスをスレッドで同時に再生する
Step 3	`step3.py`	2 チャンネルを同期させ、チャンネル間の遅延を制御する

1. ハードウェア構成

システム全体の構成を下図に示します。PC　で生成した音声信号（＝振動波形）を、**経路　A**（USB オーディオアダプタ **Sound Blaster Play! 3** 経由）または**経路 B**（PC のヘッドホン端子から直接）のどちらか一方でアンプに入力します。アンプには　**Fostex　AP05mk2**（パーソナルアンプ、5W　+ 5W）を使い、その L / R スピーカー出力に振動子を接続します。

振動提示システム ハードウェア構成

経路 A（USBオーディオ経由）または 経路 B（PC直挿し）のどちらか一方を使う



接続手順

① 経路 A の場合は Sound Blaster Play! 3 を PC の USB 端子に挿す（経路 B の場合は不要）。② 3.5mm ステレオミニケーブルで、Sound Blaster のヘッドホン端子（経路 B では PC のヘッドホン端子）と AP05mk2 の INPUT をつなぐ。③ AP05mk2 の L / R スピーカー出力端子に振動子のケーブルを接続する（振動子が 1 つのときは L のみ）。④ AP05mk2 に AC アダプターをつなぎ、VOLUME を最小にしてから電源を入れる。

> **⚠ 音量に関する注意**
> VOLUME は必ず最小から始め、振動を確認しながら少しずつ上げてください。大きすぎる出力は振動子の破損や、肌への不快な刺激の原因になります。また、プログラム側の `AMPLITUDE` は 1.0 以下で使用してください（1.0 を超えると波形が歪みます）。

2. 環境構築

Python 3 がインストールされた PC で、ターミナル（コマンドプロンプト）から次の 2 つのライブラリをインストールします。`numpy` は波形の計算に、`sounddevice` はオーディオデバイスへの出力に使います。

```
pip install numpy sounddevice
```

Step 1・2・3 は「OS の既定（デフォルト）の出力デバイス」から再生されます。Sound Blaster Play! 3 を使う場合は、OS のサウンド設定で既定の出力デバイスを Sound Blaster に切り替えておいてください（Windows: 設定 → システム → サウンド ／ macOS: システム設定 → サウンド）。

3. Step 1 — 波形を作って再生する

step1.py

```
import numpy as np
import sounddevice as sd

FREQUENCY = 250
DURATION = 2
AMPLITUDE = 0.5
SAMPLERATE = 44100

t = np.linspace(0, DURATION, int(SAMPLERATE * DURATION), endpoint=False)
wave = (AMPLITUDE * np.sin(2 * np.pi * FREQUENCY * t)).astype(np.float32)

sd.play(wave, SAMPLERATE)
sd.wait()
```

プログラムの説明

最初の 4 つの大文字の変数が「波形のデザイン」に当たります。`FREQUENCY` は振動の周波数 (Hz)、`DURATION` は長さ（秒）、`AMPLITUDE` は振幅（0.0〜1.0）、`SAMPLERATE` は 1 秒あたりのサンプル数です。250　Hz　は皮膚の振動感覚（パチニ小体）の感度が高い帯域で、振動提示によく使われます。

`np.linspace` は 0 秒から `DURATION` 秒までを `SAMPLERATE × DURATION` 個（ここでは 88,200 個）に等分した「時刻の配列」`t` を作ります。次の行で正弦波の式 A sin(2πf t) に `t` を代入し、波形の配列 `wave` を得ます。`.astype(np.float32)` はオーディオ出力で標準的な 32 bit 浮動小数点型への変換です。

最後に `sd.play` が波形を既定の出力デバイスに送って再生を開始し、`sd.wait` が再生の終了まで待ちます（`sd.wait` がないと、再生が始まった直後にプログラムが終了して音が途切れます）。実行は次のとおりです。

```
python step1.py
```

試してみよう

`FREQUENCY` を 100・200・300 に変えると、振動の「粗さ」がどう変わるか比べてみましょう。`AMPLITUDE` を 0.1 と 0.5 で比べると強度の違いが分かります。`DURATION` を 0.2 にすると短いパルス状の振動になります。

4. Step 2 ― 2 チャンネル同時再生（ステレオ）

次に、1 台のデバイスの L / R チャンネルを**スレッド（thread）**で同時に鳴らします。スレッドとは、1 つのプログラムの中で複数の処理を並行して走らせる仕組みです。「L を再生する処理」と「R を再生する処理」をそれぞれ別のスレッドに任せることで、2 チャンネルが同時に再生されます。

step2.py

```
import numpy as np
import sounddevice as sd
import threading

FREQUENCY = 250
DURATION = 2
AMPLITUDE = 0.5
SAMPLERATE = 44100

t = np.linspace(0, DURATION, int(SAMPLERATE * DURATION), endpoint=False)
wave = (AMPLITUDE * np.sin(2 * np.pi * FREQUENCY * t)).astype(np.float32)

def play(channel):
    stereo = np.zeros((len(wave), 2), dtype=np.float32)
    stereo[:, channel] = wave
    with sd.OutputStream(samplerate=SAMPLERATE, channels=2) as stream:
        stream.write(stereo)

left = threading.Thread(target=play, args=(0,))
right = threading.Thread(target=play, args=(1,))
left.start()
right.start()
left.join()
right.join()
```

プログラムの説明

波形を作るところまでは Step 1 と同じです。関数 `play(channel)` は、まず「行数 = サンプル数、列数 = 2（L と R）」のゼロ埋めされたステレオ配列を作り、指定された列（`channel` が 0 なら L、1 なら R）にだけ波形を書き込みます。もう一方の列は 0 のままなので無音です。

Step 1 の `sd.play` は同時に 1 つしか再生できないため、ここでは各スレッドが自分専用の出力ストリーム `sd.OutputStream` を開き、`stream.write` で波形を書き込みます。`with` 文はストリームの開始と後片付けを自動で行います。

最後の　6　行がスレッドの基本形です。`threading.Thread(target=play,　args=(0,))`　で「`play(0)` を実行するスレッド」を作り、`start()` で走らせ、`join()` で両方のスレッドが終わるのを待ちます。`left` と `right` はほぼ同時に走り出すため、L / R の振動子が同時に振動します。

試してみよう

`FREQUENCY` を `FREQUENCY_LEFT = 250` と `FREQUENCY_RIGHT = 100` の 2 つに分け、波形も `wave_left` と `wave_right` の 2 本作って、左右で異なる振動を提示してみましょう（`play` に波形も渡すよう変更が必要です）。

5. Step 2-2 — 2 台のオーディオデバイスで同時再生

ステレオの 2 チャンネルでは足りない場合（例: 振動子を 3 個以上使う、離れた場所に置く）、複数のオーディオデバイスを同時に使います。考え方は Step 2 と同じで、「スレッドごとに別の**デバイス**のストリームを開く」だけです。ここでは Sound Blaster Play! 3 と PC 内蔵のヘッドホン出力の 2 台を例にします。

step2_2.py

```
import numpy as np
import sounddevice as sd
import threading

DEVICE_1 = 1
DEVICE_2 = 2
FREQUENCY = 250
DURATION = 2
AMPLITUDE = 0.5
SAMPLERATE = 44100

print(sd.query_devices())

t = np.linspace(0, DURATION, int(SAMPLERATE * DURATION), endpoint=False)
wave = (AMPLITUDE * np.sin(2 * np.pi * FREQUENCY * t)).astype(np.float32)

def play(device):
    with sd.OutputStream(samplerate=SAMPLERATE, channels=1, device=device) as
stream:
        stream.write(wave)

thread_1 = threading.Thread(target=play, args=(DEVICE_1,))
thread_2 = threading.Thread(target=play, args=(DEVICE_2,))
thread_1.start()
thread_2.start()
thread_1.join()
thread_2.join()
```

プログラムの説明

`print(sd.query_devices())` は、PC に接続されているオーディオデバイスの一覧を番号付きで表示します。実行すると次のような出力が得られます（環境によって異なります）。

```
  0 Microphone (USB Audio), MME (2 in, 0 out)
> 1 Speakers (Sound Blaster Play! 3), MME (0 in, 2 out)
< 2 Headphones (Realtek Audio), MME (0 in, 2 out)
```

「out」の数が 1 以上の行が出力デバイスです。この一覧を見て、使いたい 2 台の番号を `DEVICE_1` と `DEVICE_2` に書き込み、もう一度実行してください。関数 `play(device)` は Step 2 とほぼ同じですが、`device=` で使うデバイスを指定し、モノラル（`channels=1`）で書き込む点が異なります。2 つのスレッドがそれぞれ別のデバイスを開くので、2 台が同時に鳴ります。

デバイス番号は USB の抜き挿しや PC の再起動で変わることがあります。エラー（Invalid device）が出たら、表示された一覧で番号を確認し直してください。

試してみよう

Step 2 のようにステレオ配列を書き込む形（`channels=2`）に変えれば、2 台 × L / R で最大 4 つの振動子を同時に駆動できます。

6. Step 3 ― 2 チャンネルの同期と遅延の制御

最後に、L / R の 2 チャンネルを**同期**させたうえで、右チャンネルだけを一定時間**遅らせて**提示します。刺激間の時間差（SOA・遅延）は、仮現運動や時間順序判断など振動触覚の実験で最も重要なパラメータの 1 つです。

step3.py

```
import numpy as np
import sounddevice as sd
import threading
import time

FREQUENCY = 250
DURATION = 2
AMPLITUDE = 0.5
SAMPLERATE = 44100
DELAY = 0.5

t = np.linspace(0, DURATION, int(SAMPLERATE * DURATION), endpoint=False)
wave = (AMPLITUDE * np.sin(2 * np.pi * FREQUENCY * t)).astype(np.float32)

barrier = threading.Barrier(2)

def play(channel, delay):
    stereo = np.zeros((len(wave), 2), dtype=np.float32)
    stereo[:, channel] = wave
    with sd.OutputStream(samplerate=SAMPLERATE, channels=2) as stream:
        barrier.wait()
        time.sleep(delay)
        stream.write(stereo)

left = threading.Thread(target=play, args=(0, 0))
right = threading.Thread(target=play, args=(1, DELAY))
left.start()
right.start()
left.join()
right.join()
```

プログラムの説明

グローバル変数 `DELAY` が 2 チャンネル間の遅延（秒）です。ファイル冒頭でこの値を書き換えて実行すると、L が振動した `DELAY` 秒後に R が振動します。`DELAY = 0` にすれば同時です。

ポイントは**同期**の仕組みである `threading.Barrier(2)` です。バリアとは「決めた数のスレッドが全員そろうまで、先に着いたスレッドを待たせる」仕組みです。各スレッドはストリームを開き終

えてから `barrier.wait()` に到着します。2 本のスレッドがそろった瞬間に両方が同時に通過するため、「デバイスを開く時間のばらつき」が遅延に混ざらず、正確なスタートラインがそろいます。

バリア通過後、各スレッドは自分の `delay` だけ `time.sleep` で待ってから書き込みます。`left` は 0 秒（すぐ再生）、`right` は `DELAY` 秒待ってから再生するので、2 チャンネルの開始時刻の差がちょうど `DELAY` になります。

試してみよう

`DELAY` を 0.5 → 0.2 → 0.1 → 0.05 → 0.02 と小さくしながら、左右の「ずれ」をどこまで感じられるか試してみましょう。左右の手に振動子を 1 つずつ持つと分かりやすいです。`DELAY` の符号を入れ替えたバージョン（R が先）も作ってみましょう。

補足: `time.sleep` による遅延の精度は OS のスケジューリングに依存し、数 ms 程度のばらつきがあります。より厳密な遅延が必要な実験では、波形の先頭に無音（ゼロ）を `DELAY × SAMPLERATE` サンプル挿入して 1 つのステレオ配列として再生する方法（サンプル精度）を使います。

7. トラブルシューティング

症状	確認すること
音（振動）が出ない	OS の既定出力デバイス／アンプの VOLUME と電源／ケーブルの接続
`ModuleNotFoundError`	`pip install numpy sounddevice` を実行したか
`PortAudioError: Invalid device`	デバイス番号を `sd.query_devices()` の一覧で確認し直す
振動が弱い・歪む	`AMPLITUDE`（≦1.0）とアンプの VOLUME のバランスを調整